## Django Intern Assignment: Modular Entity and Mapping System Using APIView

### **Objective**

Build a Django REST Framework backend where:

* each master entity is in a separate Django app

* each mapping is in a separate Django app

* all APIs are built using **APIView only**

* API documentation is generated using **drf-yasg**

---

**## Apps Required**

**### Master apps**

* `vendor`

* `product`

* `course`

* `certification`

**### Mapping apps**

* `vendor_product_mapping`

* `product_course_mapping`

* `course_certification_mapping`

This structure is mandatory.

---

**## Functional Scope**

The system should manage these master entities:

* Vendor

* Product

* Course

* Certification

**And these mappings:**

* Vendor → Product

* Product → Course

* Course → Certification

---

## Technical Constraints

**The intern must use:**

* Django

* Django REST Framework

* **APIView only**

* drf-yasg

**The intern must **not use**:**

* ViewSets

* ModelViewSet

* GenericAPIView

* mixins

* routers

**This helps you evaluate their actual DRF basics.**

---

## Model Requirements

### Common fields for master entities

Each of these models should have fields like:

* `id`

* `name`

* `code`

* `description`

* `is_active`

* `created_at`

* `updated_at`

### Mapping models

Each mapping model should have:

* parent foreign key

* child foreign key

* `primary_mapping`

* `is_active`

* `created_at`

* `updated_at`

### Example mapping models

* `VendorProductMapping`

* `ProductCourseMapping`

* `CourseCertificationMapping`

---

## Validation Rules

**The intern must implement:**

* required field validation

* unique `code` for master records

* duplicate mapping prevention

* valid foreign key usage

* only one `primary_mapping=True` for a parent at that level

### Examples

* same vendor and product pair cannot be inserted twice

* same product and course pair cannot be inserted twice

* same course and certification pair cannot be inserted twice

* if one vendor already has a primary product mapping, another primary mapping for that same vendor should fail

---

**## API Design Requirement**

Since only APIView is allowed, the intern should create separate APIs for:

**### Master entity APIs**

For each entity:

* List/Create API

* Detail API for Retrieve/Update/Delete

**### Mapping APIs**

For each mapping:

* List/Create API

* Detail API for Retrieve/Update/Delete

**---**

**## Suggested API Structure**

**### Vendor**

* `GET /api/vendors/` → list vendors

* `POST /api/vendors/` → create vendor

* `GET /api/vendors/<id>/` → retrieve vendor

* `PUT /api/vendors/<id>/` → update vendor

* `PATCH /api/vendors/<id>/` → partial update vendor

* `DELETE /api/vendors/<id>/` → delete vendor

**### Product**

* `GET /api/products/`

* `POST /api/products/`

* `GET /api/products/<id>/`

* `PUT /api/products/<id>/`

* `PATCH /api/products/<id>/`

* `DELETE /api/products/<id>/`

### Course

* `GET /api/courses/`

* `POST /api/courses/`

* `GET /api/courses/<id>/`

* `PUT /api/courses/<id>/`

* `PATCH /api/courses/<id>/`

* `DELETE /api/courses/<id>/`

### Certification

* `GET /api/certifications/`

* `POST /api/certifications/`

* `GET /api/certifications/<id>/`

* `PUT /api/certifications/<id>/`

* `PATCH /api/certifications/<id>/`

* `DELETE /api/certifications/<id>/`

### Vendor Product Mapping

* `GET /api/vendor-product-mappings/`

* `POST /api/vendor-product-mappings/`

* `GET /api/vendor-product-mappings/<id>/`

* `PUT /api/vendor-product-mappings/<id>/`

* `PATCH /api/vendor-product-mappings/<id>/`

* `DELETE /api/vendor-product-mappings/<id>/`

### Product Course Mapping

* `GET /api/product-course-mappings/`

* `POST /api/product-course-mappings/`

* `GET /api/product-course-mappings/<id>/`

* `PUT /api/product-course-mappings/<id>/`

* `PATCH /api/product-course-mappings/<id>/`

* `DELETE /api/product-course-mappings/<id>/`

**### Course Certification Mapping**

* `GET /api/course-certification-mappings/`

* `POST /api/course-certification-mappings/`

* `GET /api/course-certification-mappings/<id>/`

* `PUT /api/course-certification-mappings/<id>/`

* `PATCH /api/course-certification-mappings/<id>/`

* `DELETE /api/course-certification-mappings/<id>/`

**---**

**## What Should Exist Inside Each App**

Each app should contain its own:

* `models.py`

* `serializers.py`

* `views.py`

* `urls.py`

* `admin.py`

**This is important because you want to evaluate modularity.**

**---**

**## APIView Expectations**

**The intern should show they understand:**

* how `get`, `post`, `put`, `patch`, `delete` are written manually

* serializer usage inside APIView

* custom error handling

* returning correct HTTP status codes

* handling `DoesNotExist`

* manual queryset filtering where needed

---

## Filtering Requirement

Ask them to support query-param based filtering in list APIs.

### Examples

* `/api/products/?vendor_id=1`

* `/api/courses/?product_id=2`

* `/api/certifications/?course_id=3`

**They can do this either in:**

* mapping list APIs, or

* master list APIs with relation-aware filtering

**This is a good test of queryset logic.**

---

## drf-yasg Requirement

The project must include:

* Swagger UI

* ReDoc

* API method documentation for APIView methods

**Expected URLs:**

* `/swagger/`

* `/redoc/`

**Also ask them to document:**

* request body

* response schema

* query parameters for filters

* success and error responses

**This is especially useful because APIView needs more manual documentation than ViewSets.**

---

## Bonus Tasks

**Good interns can additionally implement:**

* soft delete using `is_active`

* nested response APIs

* shared abstract base model for timestamps

* custom utility function for object fetching

* seed data command

* unit tests for serializers and APIs

* Postman collection

---

## Evaluation Focus

**This assignment will help you assess:**

* Django app structuring

* DRF APIView understanding

* serializer validation

* model relationships

* URL organization without routers

* clean code practices

* error handling

* API documentation quality

---

## Final Assignment Text You Can Send

### **Assignment**: Django REST Framework API Using APIView Only

Build a modular Django backend for managing Vendors, Products, Courses, Certifications, and their mappings.

### Mandatory requirements

#### 1. Create separate Django apps for:

**Master apps**

* vendor

* product

* course

* certification

**Mapping apps**

* vendor_product_mapping

* product_course_mapping

* course_certification_mapping

#### 2. Use only:

* Django

* Django REST Framework

* APIView

* drf-yasg

#### 3. Do not use:

* ViewSets

* GenericAPIView

* mixins

* routers

#### 4. Create master models for:

* Vendor

* Product

* Course

* Certification

#### 5. Create mapping models for:

* VendorProductMapping

* ProductCourseMapping

* CourseCertificationMapping

#### 6. Implement APIs for each module:

* List

* Create

* Retrieve

* Update

* Partial Update

* Delete

#### 7. Add validations:

* required fields

* unique code

* prevent duplicate mappings

* valid foreign keys

* single primary mapping per parent

#### 8. Add API documentation using drf-yasg:

* Swagger

* ReDoc

* request/response docs

* query parameter docs

#### 9. Keep code modular:

Each app must have its own:

* model

* serializer

* views

* urls

* admin

**#### 10. Add README:**

**Include**:

* setup steps

* installed apps

* migration steps

* runserver steps

* API usage examples

**---**

**## Deliverables**

* GitHub repository

* working project

* README

* Swagger docs

* sample data or seed script

**---**

**## Acceptance Criteria**

**Submission is complete only if:**

* all required apps exist

* APIs are written using APIView only

* CRUD works for all master and mapping modules

* validations work correctly

* Swagger is accessible

* code is modular and readable

* project runs without errors